

# Labortory 1

Geometric Transformation and Histogram Equalization

ECE 7410: Image Processing

Jun 3rd 2025

| Group Member      | Student ID |
|-------------------|------------|
| Keegan Rolls      | 202004149  |
| Nicholas Philpott | 202018248  |

## Geometric Transformation

The code contained within this section can be found in its entirety in [Appendix A](#)

1. Download the test image (*im1.png*) from Brightspace under Lab 01. Read the image, convert the image to a grayscale image, and display the image.

```
imc = imread("./im1.png");

img = rgb2gray(imc);

imshow(img);
title('Image in grayscale');
```

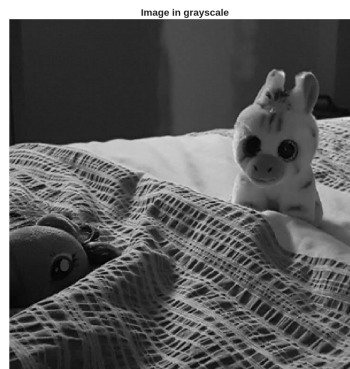


Figure 1: *img1.png* in greyscale

2. a. Define the transformation matrix with a rotation angle of 45 degrees

```
% Rotate 45 degrees
theta = 0.25*pi;

% Transformation matrix for rotation
R = [cos(theta) sin(theta) 0;
     -sin(theta) cos(theta) 0;
     0          0          1];
```

- b. Calculate the size of the transformed image and generate an empty image with the calculated size

```
[y_max, x_max] = size(img); % Obtaining the size of the image
corners = [0,      0,      1; ...
           x_max,  0,      1; ...
           0,      y_max,  1; ...
           x_max,  y_max,  1]; % Corner points of the image

new_corners = corners*R % New location of corners
new_width = round(max(new_corners(:,1)) - ...
min(new_corners(:,1)));
new_height = round(max(new_corners(:,2)) - ...
min(new_corners(:,2)));
new_img = zeros(new_height, new_width);
```

- c. Transform each pixel of the original image with the transformation matrix and assign the intensity to the new location.

*% Insert code here*

d. View the transformed image.

See **Figure 2** below

*% Insert code here*

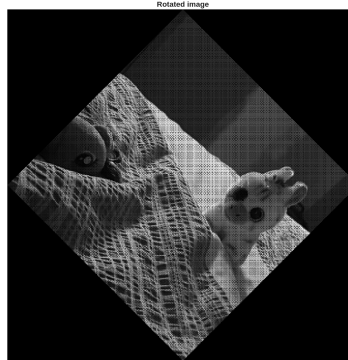


Figure 2: im1.png rotated 45 degrees

3. Complete the steps above (Q2.a - 2.d) for angle  $\theta = 90^\circ$ .

See **Figure 3** below.

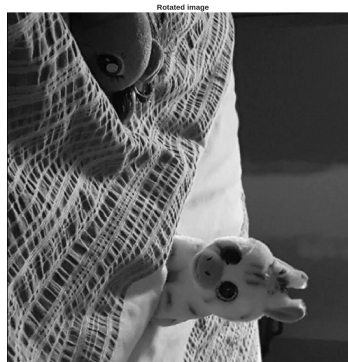


Figure 3: img1.png rotated 90°

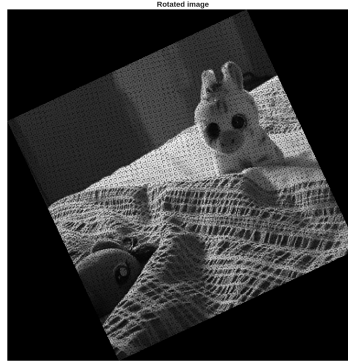
4. Complete the steps above (Q2.a - 2.d) for angle  $\theta = -25^\circ$ .

See **Figure 4** below.

5. It can be seen that in some cases, when the image is transformed the output image has ‘empty’ black pixels. Explain why this happens in only some images, but not others.

Through the process of interpolation, the image transformation will take an average of its neighbouring pixels ...

6. In the previous questions, we defined the size of the output image and mapped each input pixel to the corresponding output pixel. We can improve the output image, ensuring no ‘black’ or ‘empty’ pixels by first calculating the size of the transformed image, generating an empty image of the correct output size, and then calculate the inverse of the forward transformation matrix. For each pixel of the transformed output image, we use the inverse transformation to search

Figure 4: img1.png rotated  $-25^\circ$ 

backwards in the input image for the correct pixel. In this way, every pixel in the output image will find an input pixel. If the corresponding pixel in the input image is outside image boundary, we can simply set the value of the output pixel to zero

- a. Calculate the inverse transformation matrix.

Using the below...

$$R = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R^{-1} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Note: For a rotation matrix, the inverse is equal to its transpose. This is because rotation matrices are orthogonal matrices, where  $R^{-1} = R^T$ .

- b. Modify the code in Q2 to calculate the following inverse transformation, and use it to iterate through each output image pixel to search the input image for the following transformations. Visualize your results.
  - i. Translation ( $t_x = 50, t_y = 45$ )
  - ii. Scale ( $c_x = 0.5, c_y = 1.5$ )
  - iii. Shear (vertical) ( $s_v = 0.2$ )
  - iv. Shear (horizontal) ( $s_h = 0.3$ )
  - v. Rotation ( $\theta = 50^\circ$ ) followed by translation ( $t_x = 25, t_y = 30$ )

## Histogram Equalization

The code contained within this section can be found in its entirety in [Appendix B](#)

1. Download the test image (*im2.png*) from Brightspace under Lab 01. Read the image, convert the image to a grayscale image, and visualize it.

```
imc = imread('im2.png'); % Read the image
img = rgb2gray(imc); % Convert to grayscale
imshow(img); % View image
```

2. Grayscale images have 256 gray levels. Therefore create an empty matrix having dimensions of  $1 \times 256$ , generate the histogram and display it.

```
H=size(img,1); % Read the height of the image
W=size(img,2); % Read the width of the image
Hist_arr=zeros(1,256); % Array for holding the (original) histogram
Hist_eq_arr=zeros(1,256); % Array for holding the (equalized) histogram
```

3. Calculate  $p_r(r_k)$  for the image and display it. (Total number of pixels =  $y_{max} \times x_{max}$ ).

```
% Insert code here
```

4. Calculate the CDF from the PDF, and display it.

```
% Insert code here
```

5. Calculate the equalized histogram using equation 1 and map the original gray levels to the new gray levels. Display the original and new values.

```
% Insert code here
```

6. Display both images, before (original) and after histogram equalization

See [Figure X](#) below

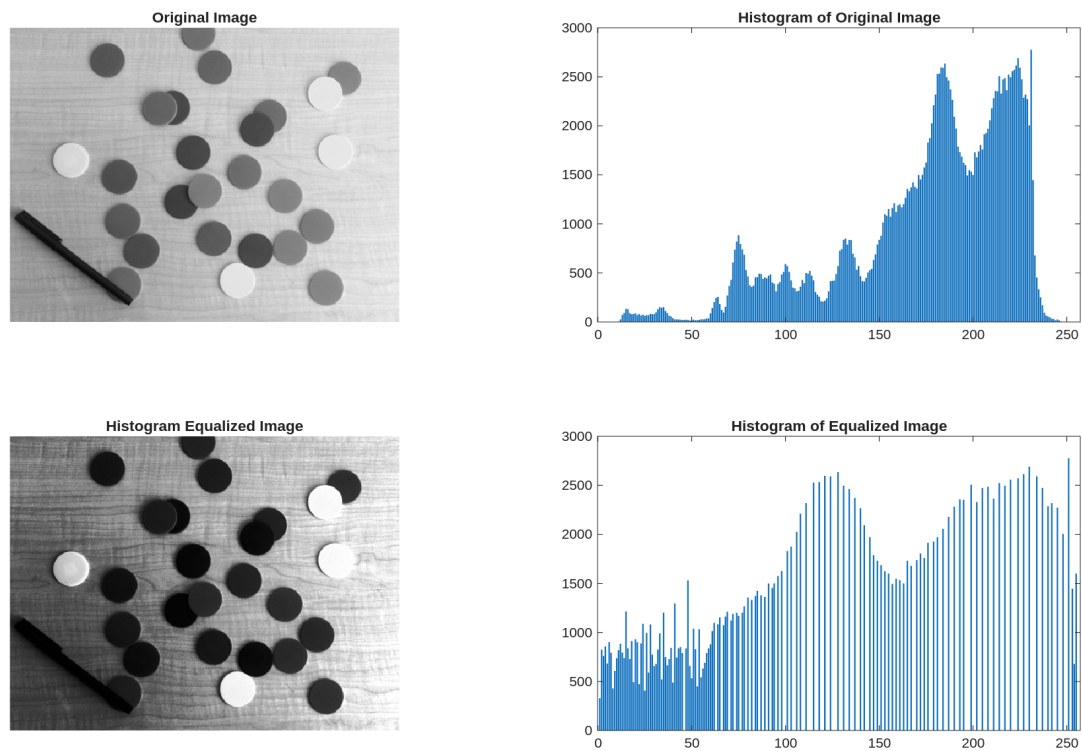


Figure 5: Before and after histogram equalization of `img2.png` with histograms

## Performance Evaluation

The code contained within this section can be found in its entirety in [Appendix C](#)

1. Download or use any image that is larger than  $640 \times 640$  pixels and use your code to rotate the image by 90 degrees. Use the inbuilt functions (e.g., `maketform` and `imtransform` in MATLAB Image Processing Toolbox) to perform the rotation of the same image. Comment on the difference in speed between the two approaches.

Figure X below shows the selected image (“im3.png”) of a digital camera on a yellow background in the first quadrant. It has dimensions  $3196 \times 3196$ .

The following code shows that the image is loaded and converted to greyscale before defining the rotation matrix.

```
% Load the image
imc = imread("./L1/im3.png");

% Convert to grayscale
img = rgb2gray(imc);

% Rotate the image by 90 degrees
theta = pi/2;

% Transformation matrix for rotation
R = [cos(theta) sin(theta) 0;
     -sin(theta) cos(theta) 0;
     0          0          1];
```

Using the same method as in [Question 1](#), a function `rotate_image` is defined. The function `rotate_image` is then timed.

```
function [rot_img] = rotate_image(img, R)
    % Rotate image `img` using the transformation matrix `R`
    [y_max, x_max] = size(img);
    fprintf('Image size: %d x %d\n', y_max, x_max);
    corners = [0, 0, 1;
               x_max, 0, 1;
               0, y_max, 1;
               x_max, y_max, 1]; % Corner points of the image

    new_corners = corners * R; % New location of corners
    new_width = round(max(new_corners(:, 1)) - min(new_corners(:, 1)));
    new_height = round(max(new_corners(:, 2)) - min(new_corners(:, 2)));
    min_width = abs(min(new_corners(:, 1)));
    min_height = abs(min(new_corners(:, 2)));
    min_width = round(min_width) + 1;
    min_height = round(min_height) + 1;

    rot_img = zeros(new_height, new_width);
    for i = 1:y_max
        for j = 1:x_max
            temp = ([j - 1, i - 1, 1]) * R;
            x_new = round(temp(1, 1)) + min_width;
            y_new = round(temp(1, 2)) + min_height;
            rot_img(y_new, x_new) = img(i, j);
        end
    end
end
```

end

```
tic; % Start timing
rot_img_custom = rotate_image(img, R);
custom_time = toc;
```

Next, the same operation is performed using inbuilt functions `maketform` and `imtransform` and timed.

```
% Rotate the image by 90 degrees using inbuilt functions
tic; % Start timing
% Create the transformation matrix using maketform
tform = maketform('affine', R);
% Apply the transformation using imtransform
rot_img_inbuilt = imtransform(img, tform);
inbuilt_time = toc; % End timing
```

Finally, **Figure 6** below was generated. As shown in **Figure 6**, the custom rotation implementation took significantly longer than using MATLAB's built-in functions. The custom implementation required iterating through each pixel individually, resulting in execution time of several seconds, while the built-in functions completed the same operation much faster due to their optimized implementation. This demonstrates the efficiency advantage of using specialized library functions for image processing tasks.

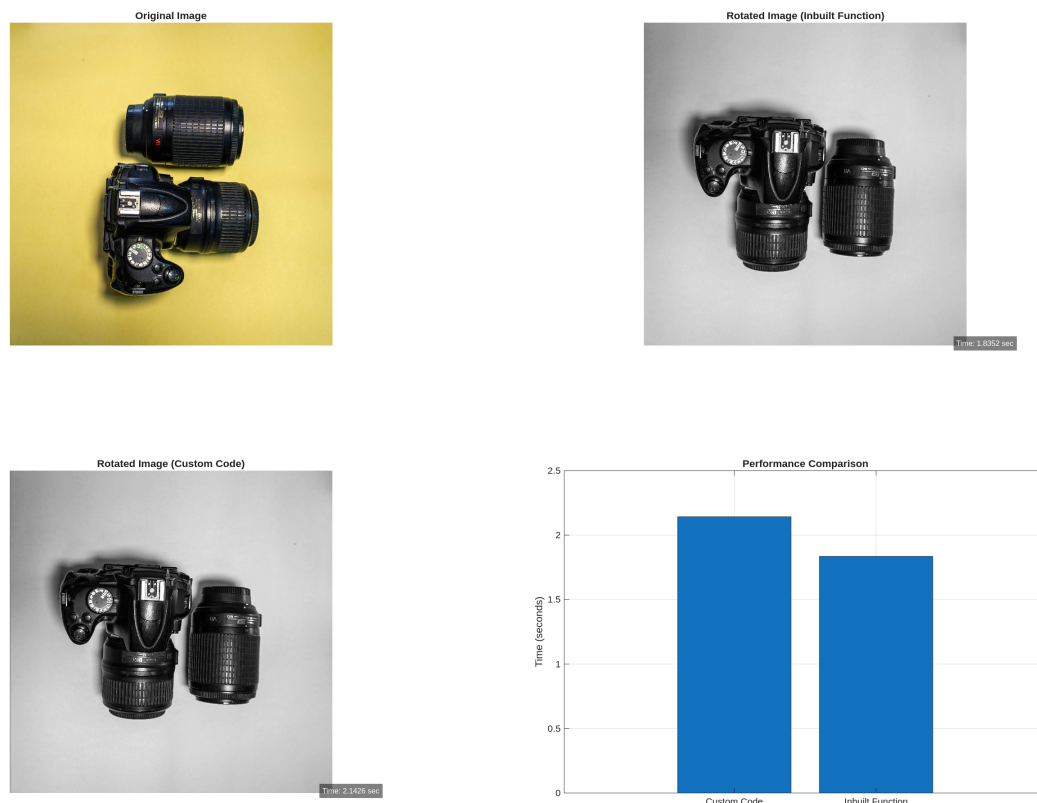


Figure 6: im3.png rotated 90° with timing comparison

2. Comment on the shape of the histogram resulting from the histogram equalization process. Is it a perfectly uniform

histogram, like we would theoretically expect? Provide an explanation of why it is, or is not, uniform.

Using the same method as [Question 2](#), “im3.png” was equalized. The below code demonstrates this process.

```
H = size(img,1); % Read the height of the image
W = size(img,2); % Read the width of the image

Hist_arr = zeros(1,256); % Array for holding the (original) histogram
Hist_eq_arr = zeros(1,256); % Array for holding the (equalized) histogram

CDF_array = zeros(1,length(Hist_arr)); % array to hold cumulative distribution function (CDF)

hist_eq_img = uint8(zeros(H,W)); % A 2D array for keeping histogram equalized image (intensity in 8 bit i

for i=1:H,
    for j=1:W,
        Hist_arr(1,img(i,j)+1)=Hist_arr (1,img(i,j)+1)+1 ;
    end
end

Hist_arr_pdf = Hist_arr/(H*W); % PDF

dummy1=0; % A dummy variable to hold the summation results
for k=1:length(Hist_arr); % Generating the CDF from PDF
    dummy1=dummy1+Hist_arr_pdf(k);
    CDF_array(k)= dummy1;
end

for l=1:H, % Histogram equalization
    for m=1:W,
        hist_eq_img(l,m)= round(CDF_array(img(l,m)+1)*(length(Hist_arr_pdf)-1)); % scale to 255 and round
        Hist_eq_arr(1, hist_eq_img(l,m)+1)=Hist_eq_arr (1,hist_eq_img(l,m)+1)+1; % Its histogram
    end
end
```

Next, [Figure X](#) was generated. The histogram resulting from the equalization process is not perfectly uniform due to several practical constraints. Digital images use discrete intensity levels (0-255 in 8-bit images), which inherently limits how evenly pixels can be redistributed, resulting in distinct spikes rather than a smooth distribution. The gaps between these spikes occur because multiple input intensity values must map to the same output value during transformation. Additionally, the original histogram shows most pixel values clustered in the higher intensity range (around 200); when redistributed, these pixels maintain their relative ordering, contributing to the non-uniformity. The discrete nature of the cumulative distribution function used in histogram equalization imposes quantization constraints that prevent perfect uniformity. Despite these limitations, the equalized histogram demonstrates significantly better utilization of the full intensity range compared to the original, with values distributed across the entire spectrum rather than concentrated in one region, which explains the improved contrast in the equalized image.

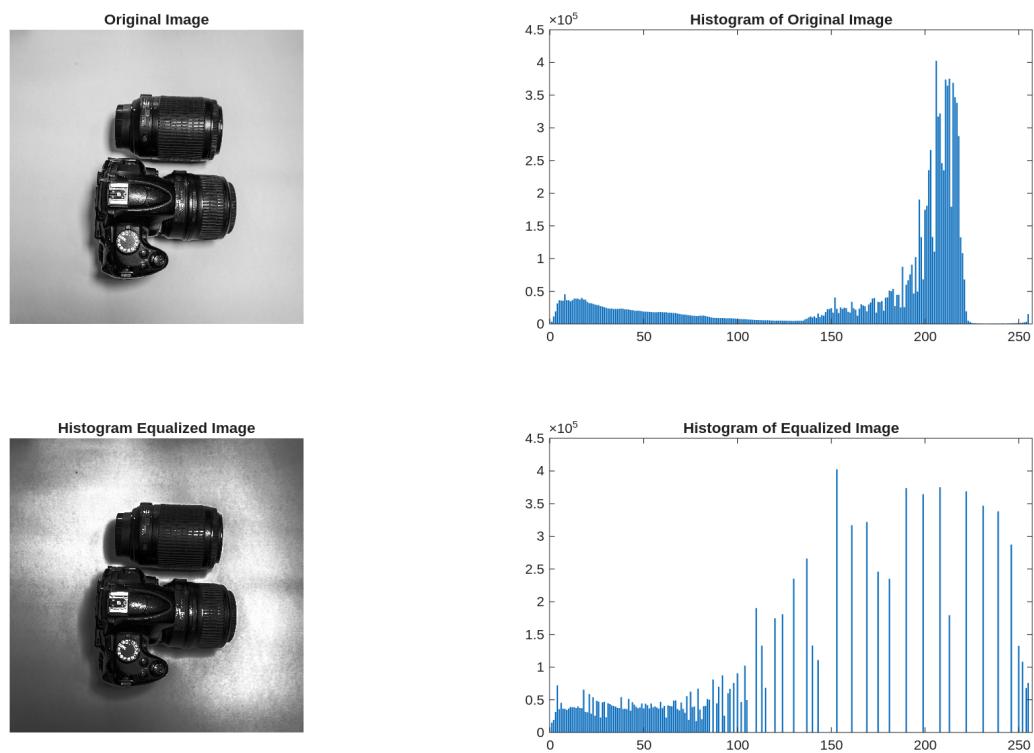


Figure 7: im3.png histogram equilization

## Appendix A

The below code is the complete code used in **Question 1**. It is also included in this submission as `question1.m`.

*% Insert code here*

## Appendix B

The below code is the complete code used in **Question 2**. It is also included in this submission as `question2.m`.

*% Insert code here*

## Appendix C

The below code is the complete code used in **Question 3**. It is also included in this submission as `question3.m`.

*% Insert code here*